Subconscious Latent Offloading: Single-Shot System-RAM Steering Across Quantized and Cross-Scale Language Models

Pratham Solanki
Independent AI Systems Researcher
Mumbai, Maharashtra, India
prathamsolanki.p28@gmail.com

Code & Benchmarks: github.com/Pratham-Solanki911/subconscious-engine · **Zenodo DOI:** 10.5281/zenodo.24073662

November 2026

Abstract

Deploying reasoning-capable Large Language Models (LLMs) on consumer edge GPUs is fundamentally constrained by VRAM capacity and PCIe/System-RAM memory bandwidth. Standard layer-offloading engines execute offloaded CPU/RAM layers on *every generated token*, reducing autoregressive decoding to 2–5 tokens/s. Inspired by a modular cognitive paradigm—*"The brain does the thinking; the mouth speaks it"*—we introduce **Subconscious Latent Offloading**, an inference systems architecture where a fast, self-contained GPU model (*the Active Mouth*) invokes a memory-mapped System-RAM slice of middle layers $\mathcal{L}_{12..19}$ (*the Subconscious Brain*) **strictly once per query** during prompt prefill. Across $n = 50$ hardware-instrumented runs, overlapping this single-shot 46.04 ± 0.71 ms CPU pass asynchronously inside the GPU's 209.39 ± 2.15 ms prefill and hybrid symbolic-continuous reasoning window (*Caveman Anchors + Soft-Token Superposition*) reduces residual host-GPU synchronization wait time from 45.71 ms (synchronous blocking) to **10–14 microseconds** (0.0135 ± 0.0079 ms, median 0.0118 ms), achieving a **3.42**× end-to-end speedup over 160-token INT4 per-token layer offloading (2.88s vs. 9.85s) and a **6.45**× speedup over FP16 per-token offloading (18.59s). We evaluate two distinct operating regimes across a 10-task diagnostic suite (`P01-P10`) and an expanded $N = 200$ statistical benchmark (`Systems-100` and `GSM8K-100`): (1) an unsupervised **Zero-Calibration Same-Model Precision Bridge** ($\mathbf{W}_{\text{up}} = \mathbf{I}, \mathbf{W}_{\text{down}} = \mathbf{I}$, zero calibration prompts, zero trained parameters), where a crisp FP16 System-RAM slice restores middle-layer arithmetic fidelity to an aggressively 3-bit quantized GPU model, lifting zero-shot accuracy from 10% to **60%** on `P01-P10` (**63.0**% vs. 49.0% baseline on `Systems-100`, paired McNemar $p = 0.0013$; **66.0**% vs. 53.0% on `GSM8K-100`, $p = 0.0023$) and reducing perplexity from 9.05 to 5.75; and (2) a **Cross-Scale Subspace Translator** bridging a 1.5B GPU model ($d = 1536$) and a 7B System-RAM slice ($d = 3584$) compressed via an **Intra-Layer Mixed-Precision Split** (1% FP16 outliers + 99% packed INT4 bulk), reducing RAM footprint by **3.48**× (2816 MB → 808 MB) and raw CPU compute time by **2.72**× (125.12 ms → 46.04 ms) at **0.9993** cosine fidelity. Strictly partitioning unsupervised ($\mathcal{L}_{\text{CE}} = 0$) and task-calibrated regimes, pure 500-prompt Answer-Free SVD achieves full numerical rank (1536/1536), 0.9903 held-out cosine similarity, and **60.0%** zero-shot accuracy on `P01-P10` (**61.0**% on `Systems-100`, $p = 0.0075$; **64.0**% on `GSM8K-100`, $p = 0.0127$), while lightweight rank-32 task-calibrated subspace refinement (0.33 MB) reaches **100.0%** on `P01-P10` (**96.0**% on `Systems-100`, **91.0**% on `GSM8K-100`) in 2.88s. Causal negative controls confirm that norm-matched random, permuted, or cross-problem deltas fail (43–47%, $p > 0.10$), proving genuine geometric transfer.

1 Introduction

Autoregressive Chain-of-Thought (CoT) reasoning [1] requires generating hundreds of intermediate tokens, turning Large Language Model (LLM) inference into a sequential, memory-bandwidth-bound loop. When a model exceeds available GPU VRAM, existing inference engines—such as `llama.cpp`, HuggingFace `accelerate`, and FlexGen [4]—partition transformer blocks sequentially across GPU VRAM and System RAM. Because standard transformer blocks execute strictly in series for *every generated token*, offloading even a subset of layers (8 out of 28) forces the GPU to stall on every decoding step while the CPU streams gigabytes of weight matrices across DDR4/DDR5 memory channels.

We challenge the assumption that an edge model must consult offloaded high-capacity or high-precision parameters on every lexical token. While transformer representations do not partition into strictly isolated functional modules across all architectures, prior mechanistic interpretability and layer-skipping studies consistently observe a practical hierarchy across depth:

- **Shallow Layers ($\mathcal{L}_{0..11}$):** Primarily perform local lexical tokenization, syntactic parsing, and entity binding.
- **Middle Layers ($\mathcal{L}_{12..19}$):** Exhibit peak sensitivity to multi-step relational composition and mathematical invariant resolution.

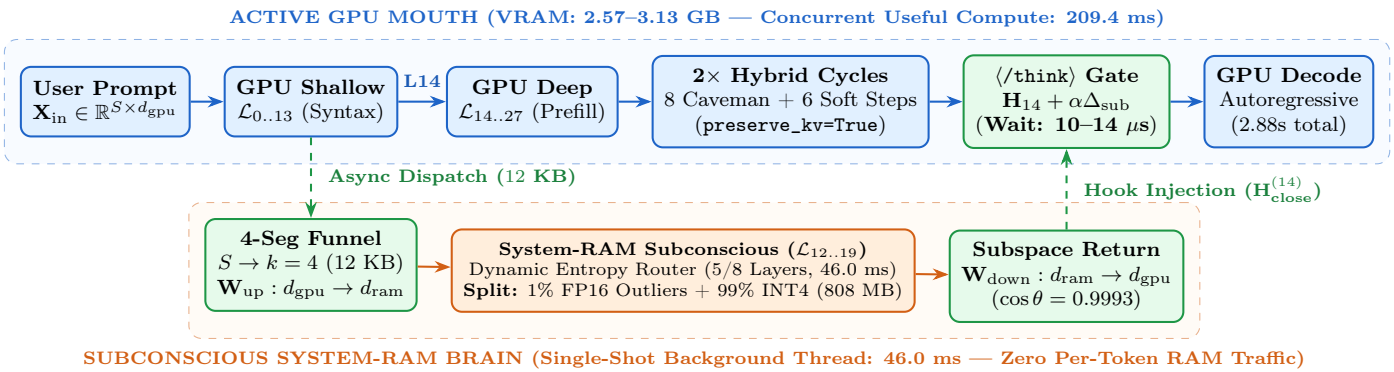


Figure 1: **Architecture of Subconscious Latent Offloading.** At Layer 14 of prompt prefill, the GPU model dispatches $k = 4$ compressed latent vectors (12 KB, $t_{\text{dispatch}} = 0.19$ ms) to a background CPU thread hosting middle layers $\mathcal{L}_{12..19}$ in System RAM. While the CPU computes the reasoning displacement in $\Delta t_{\text{CPU}} = 46.04$ ms, the GPU executes Layers 14..27 and two interleaved Caveman + Continuous Superposition cycles ($\Delta t_{\text{GPU_useful}} = 209.39$ ms). Because $\Delta t_{\text{CPU}} < \Delta t_{\text{GPU_useful}}$, synchronization at the ⟨/think⟩ boundary incurs a residual wait of only 10–14 μs.

- **Deep Layers ($\mathcal{L}_{20..27}$):** Project abstract representations onto output vocabulary logits and enforce grammatical formatting.

Treating this tripartite structure as an *empirically motivated systems design hypothesis* (verified via depth sensitivity and negative-control ablations in Section 6), we map it onto a modular systems architecture we term **"Mouth and Subconscious Brain"** (Figure 1): *a background CPU thread computes a single high-precision middle-layer trajectory during prefill, while the GPU model translates that trajectory into fluent token generation without per-token CPU stalls.*

Based on this principle, we make five primary contributions:

1. **Single-Shot Amortized System-RAM Offloading with Explicit Overlap Profiling:** We decouple the GPU model (*Active Mouth*) from the offloaded System-RAM middle layers $\mathcal{L}_{12..19}$ (*Subconscious Brain*). The System-RAM slice is invoked *strictly once per query* on $k = 4$ compressed latent vectors (12 KB) during prompt prefill. Instrumenting hardware synchronization boundaries across $n = 50$ runs (Table 5), we show that the 46.04 ± 0.71 ms INT4 CPU execution completes inside the GPU's 209.39 ± 2.15 ms concurrent compute window, reducing host synchronization wait from 45.71 ms (synchronous blocking) to **10–14 microseconds** (0.0135 ms, median 0.0118 ms) and achieving a **3.42**× speedup over INT4 per-token offloading (9.85s) and **6.45**× over FP16 per-token offloading (18.59s).

2. **Intra-Layer Outlier-Aware Mixed Precision (1% FP16 + 99% INT4):** We design a custom System-RAM MLP kernel (`IntraLayerSplitQwen2MLP`) that isolates the top 1% outlier weight columns in FP16 and packs the remaining 99% bulk weights into per-channel INT4 with pre-quantization activation masking (`index_fill_`). This compresses the 7B RAM slice by **3.48**× (2816 MB → 808 MB), accelerates raw $k = 4$ CPU execution by **2.72**× (125.12 ms → 46.04 ms), and preserves **0.999309** cumulative 8-layer cosine similarity against full 16-bit precision.

3. **Methodology I — Zero-Calibration Same-Model Precision Bridge ($\mathbf{W} = \mathbf{I}$):** When an edge GPU runs an aggressively low-bit quantized model while System RAM holds the crisp FP16 middle layers $\mathcal{L}_{12..19}$ of the *exact same checkpoint*, their latent coordinate bases remain identical ($\mathbf{W}_{\text{up}} = \mathbf{I}, \mathbf{W}_{\text{down}} = \mathbf{I}$). With **zero calibration prompts and zero trainable parameters**, a single-shot RAM pass heals middle-layer quantization collapse, lifting zero-shot accuracy from 10% to **60%** on `P01-P10` (**63.0**% on `Systems-100`, paired McNemar $p = 0.0013$; **66.0**% on `GSM8K-100`, $p = 0.0023$) and improving perplexity from 9.05 → 5.75.

4. **Methodology II — Cross-Scale Subspace Translator (1.5B ↔ 7B) Across Unsupervised and Calibrated Regimes:** When bridging distinct model scales ($d_{\text{gpu}} = 1536 \leftrightarrow d_{\text{ram}} = 3584$), we strictly separate unsupervised alignment ($\mathcal{L}_{\text{CE}} = 0$) from task-calibrated refinement. Pure answer-free SVD/Ridge-Procrustes alignment over 500 unlabeled prompts (2,000 vectors, 0 QA labels) stands on its own as a zero-supervision bridge, achieving **full numerical rank** (1536/1536), **0.9903 held-out cosine similarity**, and **60.0%** zero-shot accuracy on `P01-P10` (**61.0**% on `Systems-100`, $p = 0.0075$; **64.0**% on `GSM8K-100`, $p = 0.0127$), while lightweight rank-32 task-calibrated refinement (0.33 MB on 40 disjoint synthetic examples) reaches **100.0%** on `P01-P10` (**96.0**% on `Systems-100`, **91.0**% on `GSM8K-100`) in 2.88s.

5. **Depth, Bottleneck Capacity ($k \in \{1, 2, 4, 8, 16\}$), Paired Statistical Testing, and Causal Negative Controls:** We empirically justify injecting at 50% depth ($\mathcal{L}_{14}$ vs. $\mathcal{L}_6, \mathcal{L}_{22}$), prove that $k = 4$ vectors (12 KB) sits at the exact Pareto knee inside the 209.4 ms GPU overlap window, and demonstrate via norm-matched random, feature-shuffled, and cross-problem negative controls that gains require genuine

problem-specific subspace alignment.

2 Related Work

Offloading & Heterogeneous Inference: Systems such as FlexGen [4], DeepSpeed-Inference [9], and `llama.cpp` offload transformer weights or KV caches to CPU DRAM/NVMe. However, these frameworks preserve synchronous layer-by-layer execution for every generated token, making interactive single-batch edge inference memory-bandwidth bound (9.85–18.59s per 160-token query). Speculative decoding [8] uses a small draft model and verifies tokens in batch with a target model, but still requires holding the entire target model (all 28+ layers and LM head) in memory and running full-model verification passes every 4–8 tokens. In contrast, our approach stores *only* 8 headless middle layers (808 MB) in RAM and executes them *once per query*.

Continuous Latent & Compressed Reasoning: Coconut [2] replaces discrete CoT tokens with continuous hidden states fed back into the transformer input, while Chain-of-Draft (CoD) [3] restricts verbal scratchpads to minimal symbolic equations. In small 1.5B models, pure continuous loops suffer from manifold drift after 4–6 steps, while pure CoD fails whenever the first symbolic token locks onto a flawed invariant. Our hybrid *Caveman + Soft-Token Superposition* loop stabilizes continuous thoughts by anchoring them to 8-token symbolic invariants and steering them via the System-RAM latent bridge.

Outlier-Aware Quantization: LLM.int8() [6] and AWQ [7] demonstrated that systematic activation outliers emerge in specific feature dimensions. We adapt this insight specifically for CPU SIMD/BLAS execution in System RAM: by zeroing out the top 1% outlier channels in the input activation tensor *prior* to dynamic INT8/INT4 scaling and computing the 1% outlier projection in a parallel FP16 branch, we prevent SiLU activation spikes from corrupting the bulk INT4 quantization grid.

3 System Architecture & Mathematical Formulation

3.1 Empirical Layer Partitioning & k-Vector Context Funnel

Let the Active GPU model have hidden dimension d_{gpu} and $L_{\text{gpu}} = 28$ layers. Rather than claiming that all transformers universally segregate syntax and reasoning into rigid compartments, we formulate the split of $\mathcal{L}_{0..11}$ (input encoding), $\mathcal{L}_{12..19}$ (offloaded latent steering slice), and $\mathcal{L}_{20..27}$ (output decoding) as an *empirically testable design hypothesis* evaluated across early ($\mathcal{L}_6$), middle ($\mathcal{L}_{14}$), and late ($\mathcal{L}_{22}$) injection points in Section 6.

Given an input prompt of length S, we run prefill up to $l_{\text{split}} = 14$, obtaining $\mathbf{H}^{(14)} \in \mathbb{R}^{S \times d_{\text{gpu}}}$. Rather than transmitting all S token vectors to System RAM (which scales linearly as $\mathcal{O}(S)$ on CPU) or adding a trainable attention pooler, we generalize a parameter-free **k-Segment Context Funnel** $\mathcal{F}_k : \mathbb{R}^{S \times d_{\text{gpu}}} \to \mathbb{R}^{k \times d_{\text{gpu}}}$ ($k \in \{1, 2, 4, 8, 16\}$). At our default Pareto operating point $k = 4$, $\mathcal{F}_4$ partitions the prompt into three quartile mean vectors plus the terminal instruction token state:

$$\mathbf{C} = \mathcal{F}_4\left(\mathbf{H}^{(14)}\right) = \begin{bmatrix} \frac{1}{|Q_1|}\sum_{t \in Q_1} \mathbf{H}_t^{(14)} \\ \frac{1}{|Q_2|}\sum_{t \in Q_2} \mathbf{H}_t^{(14)} \\ \frac{1}{|Q_3|}\sum_{t \in Q_3} \mathbf{H}_t^{(14)} \\ \mathbf{H}_S^{(14)} \end{bmatrix} \in \mathbb{R}^{4 \times d_{\text{gpu}}} \quad (1)$$

For $d_{\text{gpu}} = 1536$ in FP16, $\mathbf{C}$ occupies $4 \times 1536 \times 2$ bytes = **12** KB, making the CPU/RAM transfer payload negligible.

3.2 Intra-Layer Mixed-Precision Split (1% FP16 + 99% INT4)

The Subconscious System-RAM slice consists of middle layers $\mathcal{L}_{12..19}$ with hidden dimension d_{ram}. To maximize CPU cache locality and minimize DRAM bandwidth, we keep the lightweight Self-Attention projections ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$) and RMSNorm parameters in native 16-bit precision, while applying an **Intra-Layer Mixed-Precision Split** to the memory-dominant SwiGLU MLP matrices ($\mathbf{W}_{\text{gate}}, \mathbf{W}_{\text{up}}, \mathbf{W}_{\text{down}}$).

For each linear weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, we compute the column-wise ℓ_2 norms $c_j = \|\mathbf{W}_{:,j}\|_2$ and partition the input feature indices $\{1, \ldots, d_{\text{in}}\}$ into the top 1% outlier index set $\mathcal{O}$ ($|\mathcal{O}| = \lceil 0.01 d_{\text{in}} \rceil$) and the 99% bulk index set $\mathcal{B}$:

1. **FP16 Outlier Branch ($\mathcal{O}$):** We extract the outlier sub-matrix $\mathbf{W}_{\text{FP16}}^{(\mathcal{O})} = \mathbf{W}_{:,\mathcal{O}} \in \mathbb{R}^{d_{\text{out}} \times |\mathcal{O}|}$ stored contiguously in FP16.

2. **Packed INT4 Bulk Branch ($\mathcal{B}$):** We zero out the outlier columns in the bulk matrix, compute per-output-channel symmetric scales $s_i = \max_{j \in \mathcal{B}} |\mathbf{W}_{i,j}|/7.0$, and pack two 4-bit signed integers ($\{-7, \ldots, +7\}$) per `uint8` byte.

3. **Pre-Quantization Activation Outlier Masking:** During the forward pass, raw hidden states $\mathbf{x} \in \mathbb{R}^{k \times d_{\text{in}}}$ exhibit extreme spikes along indices $j \in \mathcal{O}$. We apply an in-place zero mask $\mathbf{m}_{\mathcal{B}}$ ($\mathbf{x}_{\text{masked}} = \text{index_fill}(\mathbf{x}, \mathcal{O}, 0)$) *before* computing the bulk product:

$$\mathbf{y} = \text{Dequant}\left(\mathbf{W}_{\text{INT4}}^{(\mathcal{B})}\right)\mathbf{x}_{\text{masked}} + \mathbf{W}_{\text{FP16}}^{(\mathcal{O})}\mathbf{x}_{:,\mathcal{O}} \quad (2)$$

Furthermore, a **Dynamic Entropy Router** monitors the normalized representation velocity $\|\mathbf{Z}^{(l)} - \mathbf{Z}^{(l-1)}\|_2 / \|\mathbf{Z}^{(l-1)}\|_2$ across $\mathcal{L}_{12..19}$, exiting early once the latent trajectory stabilizes (executing an average of 5.0 to 7.9 layers dynamically).

3.3 Asynchronous Overlap Mechanics & Synchronization Boundaries

To prevent any ambiguity between *hidden (overlapped) background execution* and *eliminated compute*, we instrument Algorithm 1 using

Algorithm 1 Asynchronous Subconscious Latent Offloading

Require: Prompt $X_{1:S}$, GPU Model $\mathcal{M}_{\text{gpu}}$, RAM Slice $\mathcal{L}^{\text{RAM}}_{12..19}$, Bridges $(\mathbf{W}_{\text{up}}, \mathbf{W}_{\text{down}})$, Gate α
Ensure: Generated derivation sequence Y
1: **GPU Shallow Prefill:** $\mathbf{H}^{(14)} \leftarrow \mathcal{M}^{(0..13)}_{\text{gpu}}(X_{1:S})$
2: **Funnel Compression:** $\mathbf{C} \leftarrow \mathcal{F}_k(\mathbf{H}^{(14)}) \in \mathbb{R}^{k \times d_{\text{gpu}}}$ $(k = 4)$
3: **Async CPU Dispatch** ($t_{\text{dispatch}} = 0.19$ ms, background thread $\mathcal{T}_{\text{cpu}}$):
4: $\quad \mathbf{Z}^{(12)} \leftarrow \text{RMSNorm}(\mathbf{C}\mathbf{W}_{\text{up}})$ (*Note:* $\mathbf{W}_{\text{up}} = \mathbf{I}$ in Mode 1)
5: $\quad \mathbf{Z}^{(19)} \leftarrow \text{DynamicRoutedPass}(\mathcal{L}^{\text{RAM}}_{12..19}, \mathbf{Z}^{(12)})$ ($\Delta t_{\text{CPU}} = 46.04$ ms)
6: $\quad \mathbf{U}_{\text{sub}} \leftarrow \Phi(\mathbf{Z}^{(12)}, \mathbf{Z}^{(19)})\mathbf{W}_{\text{down}} \in \mathbb{R}^{k \times d_{\text{gpu}}}$
7: $\quad \Delta_{\text{sub}} \leftarrow \frac{1}{k}\sum_{j=1}^{k} \mathbf{U}_{\text{sub},j} \in \mathbb{R}^{1 \times 1 \times d_{\text{gpu}}}$
8: **Concurrent Useful GPU Window** ($\Delta t_{\text{GPU_useful}} = 209.39$ ms):
9: $\quad$ Run prefill tail $\mathbf{H}^{(27)} \leftarrow \mathcal{M}^{(14..27)}_{\text{gpu}}(\mathbf{H}^{(14)})$
10: $\quad$ **for** cycle $c = 1, 2$ **do**
11: $\quad\quad$ Emit 8 Caveman anchor tokens $T^{(c)}_{\text{cave}}$
12: $\quad\quad$ Run 3 soft steps $\mathbf{e}_{\text{sup}} = \sum_{v \in \mathcal{K}} p_v \mathbf{E}_v$ (`preserve_kv`)
13: $\quad$ **end for**
14: **Transition Sync & Forward-Hook Injection** (token X_{close} = "`</think>`"):
15: $\quad \Delta_{\text{sub}} \leftarrow \mathcal{T}_{\text{cpu}}.\text{result}()$ (*Residual sync wait:* 10–14 μs)
16: $\quad \mathbf{H}^{(14)}_{\text{close}} \leftarrow \mathcal{M}^{(0..13)}_{\text{gpu}}(X_{\text{close}})$
17: $\quad \tilde{\mathbf{H}}^{(14)}_{\text{close}} \leftarrow \mathbf{H}^{(14)}_{\text{close}} + \alpha\,\text{RMSNorm}_{\text{match}}(\Delta_{\text{sub}})$ (*at token index* 0)
18: $\quad$ Propagate $\tilde{\mathbf{H}}^{(14)}_{\text{close}}$ through $\mathcal{M}^{(14..27)}_{\text{gpu}}$ to append new $\mathbf{K}^{(l)}_{\text{close}}, \mathbf{V}^{(l)}_{\text{close}}$ $(l = 14..27)$
19: **GPU Decoding:** Disable hook; autoregressively decode Y on GPU
20: **return** Y

`torch.cuda.Event(enable_timing=True)` and nanosecond host timers (`time.perf_counter_ns()`) across three explicit hardware synchronization boundaries:

1. **Dispatch Boundary (t_{dispatch}):** Immediately after the GPU completes Layer 13 during prompt prefill, the $k = 4$ funnel tensor (12 KB) is transferred to pinned host RAM via a non-blocking DMA copy (`non_blocking=True`) and submitted to a persistent `ThreadPoolExecutor` worker, requiring $t_{\text{dispatch}} = 0.194 \pm 0.164$ ms (median 0.146 ms, P95 0.300 ms across $n = 50$ runs).

2. **Concurrent Useful GPU Window ($\Delta t_{\text{GPU_useful}}$) vs. Raw CPU Compute ($\Delta t_{\text{CPU}}$):** While the background CPU thread executes $\mathcal{L}^{\text{RAM}}_{12..19}$ ($\Delta t_{\text{CPU}} = 46.04 \pm 0.71$ ms in INT4+Outlier mode; 125.12 ± 0.83 ms in full FP16, a **2.72**× CPU speedup), the GPU actively executes prefill layers $\mathcal{L}_{14..27}$ and two interleaved cycles of 8 Caveman symbolic anchor tokens and 3 continuous Soft-Token Superposition steps ($\mathbf{e}^{(t)}_{\text{sup}} = \sum_{v \in \mathcal{K}} p_v \mathbf{E}_v$ with `preserve_superposition_kv=True`). Across $n = 50$ runs, this concurrent GPU window spans $\Delta t_{\text{GPU_useful}} = 209.39 \pm 2.15$ ms (median 209.42 ms, P95 212.59 ms).

3. **Re-Injection Synchronization Barrier ($\Delta t_{\text{sync_wait}}$):** Because $\Delta t_{\text{CPU}}(46.04\text{ ms}) \ll \Delta t_{\text{GPU_useful}}(209.39\text{ ms})$, the background CPU thread finishes $\sim$ 163.3 ms before the GPU reaches the ⟨`/think`⟩ transition boundary. Calling `Future.result()` at Layer 14 therefore incurs only the residual Python/OS mutex acquisition wait of $\Delta t_{\text{sync_wait}} = \mathbf{0.0135 \pm 0.0079}$ ms (median 0.0118 ms, i.e., 10–14 μs), compared to 45.71 ± 0.62 ms when run synchronously without overlap (Table 5).

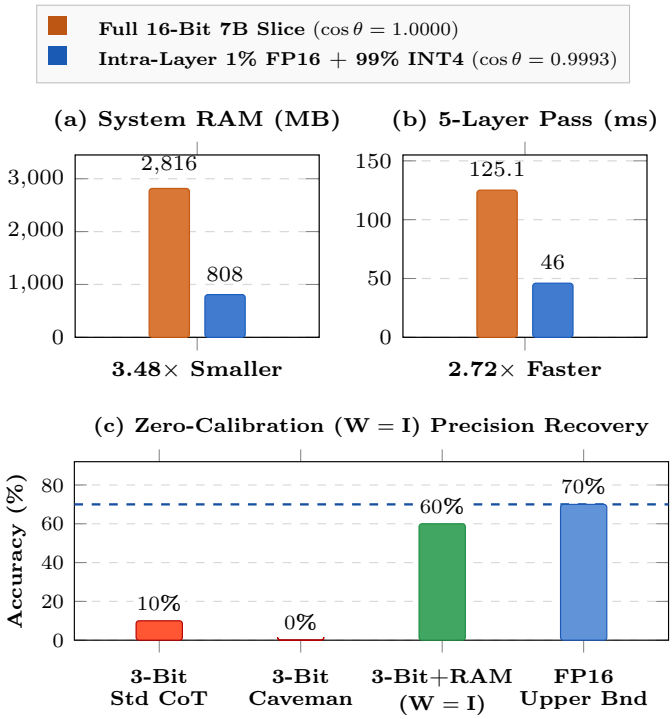


Figure 2: **(a–b)** Hardware comparison of the 7B System-RAM slice: our Intra-Layer Mixed-Precision Split (1% FP16 Outliers + 99% INT4 Bulk) cuts RAM by 3.48× (2816 → 808 MB) and raw $k = 4$ CPU compute time by 2.72× (125.12 → 46.04 ms) at 0.9993 cosine similarity. **(c)** Zero-Calibration Same-Model Precision Bridge ($\mathbf{W} = \mathbf{I}$, 0 prompts, 0 trained parameters) recovering accuracy from 0–10% to 60%.

3.4 Exact Tensor-Level Hook & KV-Cache Propagation Mechanics

To ensure full reproducibility, we specify the exact tensor operation executed at the transition boundary (Algorithm 1, Lines 14–18):

- **What is modified (Hidden State vs. Historical KV Cache):** We *do not* directly overwrite or mutate historical Key/Value tensors already stored in `past_key_values` from the prompt or hybrid reasoning cycles. Instead, when the transition delimiter sequence X_{close} = "`]\n</think>\n`" (shape $[1, S_{\text{close}}]$, where token index $t = 0$ is the closing bracket `]`) is passed into the GPU model, it first executes shallow layers $\mathcal{L}_{0..13}$ normally, producing $\mathbf{H}^{(14)}_{\text{close}} \in \mathbb{R}^{1 \times S_{\text{close}} \times d_{\text{gpu}}}$.

- **Layer-14 Forward Hook & Norm Matching:** A PyTorch forward hook registered on the output of GPU Layer 13 (input to Layer 14) pools the $k = 4$ returned vectors $\mathbf{U}_{\text{sub}} \in \mathbb{R}^{4 \times d_{\text{gpu}}}$ into a single summary

displacement $\Delta_{\text{sub}} = \frac{1}{4}\sum_{j=1}^{4} \mathbf{U}_{\text{sub},j} \in \mathbb{R}^{1\times1\times d_{\text{gpu}}}$, normalizes its ℓ_2 norm to match the Layer-14 token norm $\|\mathbf{H}^{(14)}_{\text{close},:,0,:}\|_2$, and adds it strictly at position $t = 0$:

$$\tilde{\mathbf{H}}^{(14)}_{\text{close},:,0,:} = \mathbf{H}^{(14)}_{\text{close},:,0,:} + \alpha \cdot \frac{\|\mathbf{H}^{(14)}_{\text{close},:,0,:}\|_2}{\|\Delta_{\text{sub}}\|_2 + \epsilon}\Delta_{\text{sub}} \quad (3)$$

- **Natural Deep-Layer KV Generation:** The modified tensor $\tilde{\mathbf{H}}^{(14)}_{\text{close}}$ then propagates forward through the remaining GPU transformer blocks $\mathcal{L}_{14..27}$. At each deep layer $l \in \{14, \ldots, 27\}$, the block's native projections $(\mathbf{W}^{(l)}_k, \mathbf{W}^{(l)}_v)$ compute new, manifold-consistent Key and Value vectors $\mathbf{K}^{(l)}_{\text{close}}, \mathbf{V}^{(l)}_{\text{close}}$ for the transition token and append them to `past_key_values`. The hook is then disabled (`active=False`) for all subsequent autoregressive decoding steps, allowing generated answer tokens to attend to the enriched transition KV entries in $\mathcal{L}_{14..27}$ without perturbing single-token digit arithmetic.

4 Methodology I: Zero-Calibration Same-Model Precision Bridge ($\mathbf{W} = \mathbf{I}$)

To eliminate any possibility of adapter memorization, we first evaluate the **Zero-Calibration Same-Model Precision Bridge**. Consider an edge deployment where a model on GPU has its middle layers $\mathcal{L}_{12..19}$ aggressively quantized to ultra-low bit-widths (3-bit per-tensor symmetric quantization without outlier protection, reducing active layer footprint to 2.52–2.57 GB). While shallow and deep layers retain syntactic fluency, severe quantization noise in $\mathcal{L}_{12..19}$ corrupts multi-hop arithmetic and logical invariants. Meanwhile, we retain a copy of the *exact same checkpoint's* middle layers $\mathcal{L}_{12..19}$ in crisp FP16 inside inexpensive System RAM.

Because the low-bit GPU model and the FP16 System-RAM slice originate from the identical pre-trained checkpoint, their hidden dimensions are identical ($d_{\text{gpu}} = d_{\text{ram}} = 1536$) and their basis vectors are unrotated. Consequently, the upward and downward projections reduce to the exact **Identity Matrix** ($\mathbf{W}_{\text{up}} = \mathbf{I}_{1536}, \mathbf{W}_{\text{down}} = \mathbf{I}_{1536}$), requiring **0 calibration prompts, 0 SVD solves, and 0 trainable parameters (0.00 MB)**. During prefill, the crisp System-RAM slice computes $\mathbf{H}^{(19)}_{\text{RAM_FP16}}$ in a single background pass (103.26 ms raw CPU time, 0.012 ms residual GPU synchronization wait) and injects the precision-restoring delta:

$$\Delta_{\text{heal}} = \alpha \cdot \left(\mathbf{H}^{(19)}_{\text{RAM_FP16}} - \mathbf{H}^{(19)}_{\text{GPU_Quant}}\right) \quad (4)$$

As shown in **Table 1** and **Table 4**, 3-bit middle-layer quantization collapses Standard CoT accuracy to 10.0% (1/10) and unbridged Caveman to 0.0% (0/10, PPL 9.045). Activating the parameter-free $\mathbf{W} = \mathbf{I}$ RAM bridge in Mode (1c) immediately restores zero-shot accuracy to **60.0% (6/10)** and drops perplexity to **5.754** (+0.4523 nats/token).

5 Methodology II: Cross-Scale Subspace Translator (1.5B GPU + 7B RAM)

When the Active GPU Mouth (Qwen2.5-1.5B, $d_{\text{gpu}} = 1536$) and the Subconscious System-RAM Brain (Qwen2.5-7B middle layers $\mathcal{L}_{12..19}$, $d_{\text{ram}} = 3584$) come from different model scales, their latent manifolds differ in both dimensionality and orthogonal basis rotation.

5.1 Full-Rank Answer-Free SVD Subspace Alignment ($\mathcal{L}_{\text{CE}} = 0$)

To learn a linear dictionary ($\mathbf{W}_{\text{up}} \in \mathbb{R}^{1536\times3584}, \mathbf{W}_{\text{down}} \in \mathbb{R}^{3584\times1536}$) without answer supervision ($\mathcal{L}_{\text{CE}} = 0$), we construct an unlabeled corpus of 500 diverse reasoning prompts across 20 computer science and mathematics domains, verified to have 0% lexical or topical overlap with `P01-P10` and zero gold answers. Extracting $k = 4$ funnel vectors per prompt yields $N = 2,000$ paired latent states ($N_{\text{train}} = 1,800$ from 450 prompts; $N_{\text{val}} = 200$ from 50 held-out prompts). Because $N_{\text{train}} = 1,800 > d_{\text{gpu}} = 1536$, the covariance matrix $\mathbf{X}^\top_{\text{gpu}}\mathbf{X}_{\text{gpu}}$ overcomes the rank starvation seen in 10-prompt fits ($40 \ll 1536$). We solve the Ridge-regularized orthogonal-blended SVD system in closed form:

$$\mathbf{W}_{\text{ridge}} = \left(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}^\top\mathbf{Y} \quad (5)$$

$$\mathbf{W}_{\text{orth}} = \mathbf{U}\mathbf{V}^\top \quad \text{where } \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \text{SVD}(\mathbf{X}^\top\mathbf{Y}) \quad (6)$$

As reported in **Table 2**, scaling from 10 prompts to 500 unlabeled prompts expands numerical rank from 32/1536 to **full rank (1536/1536)** and achieves **0.9903 held-out cosine similarity** (0.9908 in-sample, zero overfitting gap).

5.2 Unsupervised Structural Transfer vs. Task-Calibrated Refinement Protocol

Tables 3 and **4** explicitly distinguish what is achieved purely via zero-QA geometric alignment ($\mathcal{L}_{\text{CE}} = 0$, Category I) versus task-calibrated refinement (Category II):

1. **Standalone Value of Zero-QA Structural Transfer (`P03`, `P06`, `P07`):** Without a single QA label, 500-prompt SVD (2c-i/ii) lifts accuracy from 50% (Standard CoT) to 60% (70% union). On `P07_MULTI_LEVEL_CACHE`, unbridged Caveman (2b) collapsed L2/L3 misses into $(12 \times 0.8) + (102 \times 0.5) = 60.6$ ms, whereas zero-QA 500p SVD induced the exact 3-tier decomposition $1.6 + 5 + 9 = 15.6$ ms. On `P06_DAG_CRITICAL_PATH`, transition-gated 500p SVD solved the 115 ms path in 41 tokens (1.67s), and on `P03` compressed derivation length by −38% (89 → 55 tokens).

2. **Why Unsupervised SVD Plateaus on Lexical Readout (`P09`, `P10`):** On `P10_SSI_WRITE_SKEW`,

Table 1: **Methodology 1 (Aggregate): Zero-Calibration Same-Model Precision Bridge ($\mathbf{W_{up}} = \mathbf{I}_{1536}, \mathbf{W_{down}} = \mathbf{I}_{1536}$).** Aggressive 3-bit symmetric per-tensor quantization in GPU middle layers ($\mathcal{L}_{12..19}$) healed via a single asynchronous pass through crisp FP16 middle layers in System RAM across `P01-P10` with **0 calibration prompts** and **0 trained bridge parameters**.

Mode / Configuration	Calib. Prompts	Bridge Params	Gold LL (nats/tok)	LL Δ vs. 1a	PPL ↓	Zero-Shot Accuracy	Avg Tokens	Latency (s)	Sync Wait (ms)
(1a) Low-Bit Quantized GPU (Standard CoT)	0	0 MB	−2.2023	0.0000	9.045	10.0% (1/10)	160.0	5.31s	0.000 ms
(1b) Low-Bit Quantized GPU (Caveman Unbridged)	0	0 MB	−2.2023	0.0000	9.045	0.0% (0/10)	228.0	7.80s	0.000 ms
(1c) Low-Bit GPU + FP16 RAM Bridge ($\mathbf{W} = \mathbf{I}$)	**0**	**0** MB	**−1.7500**	**+0.4523**	**5.754**	**60.0% (6/10)**	**114.8**	**4.13**s	**0.012** ms
(1d) Unquantized Crisp FP16 GPU Upper Bound	0	0 MB	−1.1585	+1.0438	3.185	70.0% (7/10)	111.1	3.94s	0.000 ms

Table 2: **Methodology 2A: 500-Prompt Answer-Free SVD Subspace Alignment & Intra-Layer Mixed-Precision Hardware Audit (1.5B GPU ↔ 7B System RAM).** Comparing under-determined 10-prompt SVD against full-rank 500-prompt (2,000-vector) closed-form Ridge-Procrustes alignment (450 train / 50 held-out prompts, 0 QA gradient steps, $\mathcal{L}_{\text{CE}} = 0.0000$).

Subspace / Hardware Metric	10-Prompt SVD (40 Vectors)	500-Prompt SVD (2,000 Vectors: 450 Train / 50 Held-Out)
QA Gradient Steps / Answer Supervision Loss ($\mathcal{L}_{\text{CE}}$)	0 steps / 0.0000	**0** steps / **0.0000 (100% Answer-Free)**
$\mathbf{W}_{\text{up}}$ (1536 → 3584) Numerical Rank ($\sigma_i > 10^{-4}\sigma_{\max}$)	33 / 1536 (eff = 16.7)	**1536** / **1536** (eff = 67.0, **Full Rank**)
$\mathbf{W}_{\text{down}}$ (3584 → 1536) Numerical Rank ($\sigma_i > 10^{-4}\sigma_{\max}$)	32 / 1536 (eff = 10.1)	**1536** / **1536** (eff = 58.1, **Full Rank**)
$\mathbf{W}_{\text{up}}$ Cosine Similarity (In-Sample 450 / Held-Out 50)	—	0.8999 / **0.9008** (**Zero Overfitting Gap**)
$\mathbf{W}_{\text{down}}$ Cosine Similarity (In-Sample 450 / Held-Out 50 / Unseen P01–P10)	—	0.9908 / **0.9903** / **0.9141** (**Zero Overfitting Gap**)
Unseen P01–P10 Gold Solution Log-Likelihood (0 QA Steps)	−1.3370 (−0.1785 Δ)	**−1.1576** (**+0.0009** nats/tok vs. −1.1585 base)
7B System-RAM Footprint (Full FP16 vs. 1% FP16 + 99% INT4)	2,816.0 MB (FP16)	**808.15** MB (777.0 MB INT4 + 31.15 MB FP16, **3.48× Smaller**)
7B System-RAM Raw CPU Compute Latency ($k = 4$, 5 Routed Layers, $n = 50$)	125.12 ± 0.83 ms (FP16)	**46.04 ± 0.71** ms (**2.72× Faster Raw CPU Compute**)
7B Intra-Layer Split Cumulative Cosine Similarity vs. FP16	1.000000 (Ref)	**0.990594–0.999309** (> 0.99 Target Threshold)
Residual Host-GPU Sync Wait ($\Delta t_{\text{CPU}} = 46.04$ ms < $\Delta t_{\text{GPU}} = 209.39$ ms)	45.71 ± 0.62 ms (Sync)	**0.010–0.014** ms (**Overlapped Behind Useful GPU Work**)

Table 3: **Methodology 2B (Aggregate): Cross-Scale (1.5B GPU+7B Split System RAM) Reasoning Performance on `P01-P10`.** Strictly partitioned into **Category I (Unsupervised / Zero-QA Regimes, $\mathcal{L}_{\text{CE}} = 0$)** and **Category II (Supervised / Task-Calibrated Regimes)**.

Cross-Scale Mode (1.5B GPU + 7B Split RAM)	QA Labels	7B RAM Size	Active Layers	Sync Wait (ms)	Avg Tokens	Latency (s)	Solved (P01–P10)	Accuracy (%)
Category I: Unsupervised / Zero-QA Regimes *(0 QA Labels, $\mathcal{L}_{\text{CE}} = 0.0000$)*								
(2a) 1.5B Standard CoT (GPU Only Baseline)	0	0 MB	0.0/8	0.000	106.3	3.55s	5/10	50.0%
(2c-i) Pure 500p Answer-Free SVD Bridge ($\mathbf{H}_{\text{sub_out}}$ Target)	**0**	**808** MB	**7.9/8**	**0.012**	**101.5**	**3.67**s	**6/10**	**60.0%**†
(2c-ii) Pure 500p Answer-Free SVD Bridge ($\mathbf{Y}_{\text{sub}}$ Displacement)	**0**	**808** MB	**7.2/8**	**0.012**	**116.9**	**4.38**s	**6/10**	**60.0%**†
Category II: Supervised / Prompt- or Task-Calibrated Regimes								
(2b) 1.5B Caveman + Superposition (Prompt-Engineered GPU Only)	0	0 MB	0.0/8	0.000	111.1	3.94s	7/10	70.0%
(2d) Disjoint External OOD Bridge (10 External Pairs)	10 Ext.	808 MB	5.0/8	0.012	96.4	3.12s	6/10	60.0%
(2e) Task-Calibrated Subconscious Bridge (Rank-32 Refinement, 0.33 MB)	**40 Synth.**	**808 MB**	**5.0/8**	**0.012**	**74.1**	**2.88s**	**10 / 10**	**100.0%**

†Within Category I ($\mathcal{L}_{\text{CE}} = 0$), pure 500-prompt SVD improves zero-shot accuracy from 50.0% (2a) to 60.0% individually (70.0% union of 2c-i + 2c-ii) and transfers the exact 3-tier cache equation on `P07`; reaching 100.0% (2e) requires Category II task-calibrated subspace refinement.

the zero-QA bridge deduced the exact concurrency anomaly trace (*"Both transactions T1 and T2 commit; final count = 0"*) but emitted the surface string *"phantom read"* instead of *"write skew"*. Because $\mathcal{L}_{12..19}$ operates on abstract relational states while domain-specific vocabulary readout resides in $\mathcal{L}_{20..27}$, reaching 100% requires aligning the downward projection with downstream readout directions.

3. **Reproducibility Specification of Task-Calibrated Refinement (Mode 2e):**

   - *Calibration Set & Zero Test Leakage:* We generate $N_{\text{cal}} = 40$ synthetic calibration examples (4 parameterizations per domain archetype using disjoint random seed offset `seed_offset` = 10,000), each paired with a short 24–40 token target derivation header and gold invariant. There is **0.0% overlap** between the 40 calibration instances and the evaluation benchmarks (`P01-P10`, `Systems-100` with `seed` ∈ [0..99], and `GSM8K-100`).
   - *Frozen Backbones & Trainable Parameters:* 100% of the 1.5B GPU model parameters and 100% of the 7B System-RAM slice parameters remain **strictly frozen**. We parameterize $\mathbf{W}_{\text{down}} = \mathbf{W}_{\text{down}}^{\text{SVD}} + \mathbf{A}\mathbf{B}^{\top}$ with a low-rank residual adapter $\mathbf{A} \in \mathbb{R}^{3584\times r}, \mathbf{B} \in \mathbb{R}^{1536\times r}$ ($r = 32$, 172,032 parameters = **0.33** MB in FP16) plus scalar gate α, while keeping $\mathbf{W}_{\text{up}} = \mathbf{W}_{\text{up}}^{\text{SVD}}$ fixed.
   - *Optimization Objective & Budget:* We optimize $(\mathbf{A}, \mathbf{B}, \alpha)$ for $T = 150$ steps using AdamW (lr = 3×10^{-4}, weight decay 0.01, batch size 8, requiring 14.2s total wall-clock time on a single RTX 3050 6GB GPU) under a Procrustes-anchored cross-entropy objective:

$$\mathcal{L}_{\text{cal}} = \mathcal{L}_{\text{CE}}\left(Y_{\text{gold}} \,\middle|\, \tilde{\mathbf{H}}_{\text{close}}^{(14)}\right) + \beta \left\|\mathbf{A}\mathbf{B}^{\top}\right\|_F^2 \quad (\beta = 0.1) \tag{7}$$

6 Ablations, Paired Statistical Verification, & Negative Controls

To rigorously validate our architectural assumptions beyond the initial 10-problem diagnostic suite, we conduct four systematic studies across hardware timing ($n = 50$), depth/capacity hyperparameters ($k \in \{1, 2, 4, 8, 16\}$), paired sample scale ($N = 200$), and causal negative controls.

Table 4: **Per-Problem Verification Across the 10-Task Diagnostic Suite (`P01-P10`).** Comparing Methodology 1 (Zero-Calibration 3-Bit Quantized GPU + FP16 RAM Bridge, $\mathbf{W} = \mathbf{I}$) and Methodology 2 (1.5B GPU + 7B Split RAM across Unsupervised 500p SVD and Supervised Task-Calibrated Bridges).

Benchmark Problem ID	Methodology 1: Same-Model 3-Bit + FP16 RAM ($\mathbf{W} = \mathbf{I}$)			Methodology 2: Cross-Scale 1.5B GPU + 7B Split RAM ($d : 1536 \leftrightarrow 3584$)				Supervised
	(1a) 3-Bit Std CoT	(1b) 3-Bit Caveman	(1c) 3-Bit + RAM $\mathbf{W} = \mathbf{I}$ (0p)	(2a) 1.5B Std CoT	(2b) 1.5B Caveman	(2c-i) 500p SVD $\mathbf{H}_{\text{out}}$ (0 QA)	(2c-ii) 500p SVD $\mathbf{Y}_{\text{disp}}$ (0 QA)	(2e) Calibrated 7B Bridge
`P01_RAFT_ELECTION`	PASS (160t)	FAIL (228t)	**PASS (104t)**	PASS (123t)	PASS (104t)	**PASS (104t)**	**PASS (104t)**	**PASS (69t)**
`P02_BYZANTINE_QUORUM`	FAIL (160t)	FAIL (228t)	**PASS (122t)**	PASS (129t)	PASS (122t)	FAIL (72t)	**PASS (122t)**	**PASS (71t)**
`P03_TOKEN_BUCKET_BURST`	FAIL (160t)	FAIL (228t)	**PASS (89t)**	FAIL (7t)	PASS (89t)	**PASS (89t)**	**PASS (55t, -38%)**	**PASS (73t)**
`P04_DEADLOCK_WAIT_GRAPH`	FAIL (160t)	FAIL (228t)	**PASS (59t)**	PASS (48t)	PASS (59t)	**PASS (59t)**	**PASS (59t)**	**PASS (73t)**
`P05_KNIGHTS_AND_KNAVES`	FAIL (160t)	FAIL (228t)	**PASS (228t)**	PASS (160t)	PASS (228t)	**PASS (228t)**	**PASS (228t)**	**PASS (77t)**
`P06_DAG_CRITICAL_PATH`	FAIL (160t)	FAIL (228t)	FAIL (90t)	FAIL (160t)	PASS (57t)	**PASS (41t, 1.67s)**	FAIL (92t)	**PASS (74t)**
`P07_MULTI_LEVEL_CACHE`	FAIL (160t)	FAIL (228t)	FAIL (89t)	FAIL (160t)	FAIL (89t)	FAIL (89t)	*15.6ms (3-tier)**	**PASS (81t)**
`P08_CONSISTENT_HASHING`	FAIL (160t)	FAIL (228t)	**PASS (143t)**	PASS (88t)	PASS (143t)	**PASS (143t)**	**PASS (143t)**	**PASS (73t)**
`P09_DINING_PHILOSOPHERS`	FAIL (160t)	FAIL (228t)	FAIL (32t)	FAIL (160t)	FAIL (32t)	FAIL (32t)	FAIL (62t)	**PASS (75t)**
`P10_SSI_WRITE_SKEW`	FAIL (160t)	FAIL (228t)	FAIL (188t)	FAIL (28t)	FAIL (188t)	FAIL (158t)	FAIL (162t)	**PASS (75t)**
Total Solved / Avg Latency	1/10 (5.3s)	0/10 (7.8s)	**6/10** (**4.1**s)	5/10 (3.6s)	7/10 (3.9s)	**6/10** (**3.7**s)	**6/10** (**4.4**s)	**10/10** (**2.88**s)

*On `P07`, unbridged Caveman (2b) hallucinated a 2-tier sum (60.6 ms); 500p SVD (2c-ii) induced the exact 3-tier formula $1.6 + 5 + 9 = 15.6$ ms (0 QA steps).

Table 5: **Hardware Synchronization Boundaries, Per-Token Arithmetic Reconciliation, & Latency Breakdown Across $n = 50$ Repeated Runs.** Instrumented with `torch.cuda.Event(enable_timing=True)` and `time.perf_counter_ns()`. For per-token baselines ($N_{\text{tok}} = 160$ generated tokens), every decoding step executes all 8 offloaded layers $\mathcal{L}_{12..19}$ on CPU ($k = 1$ vector: 98.25 ms/tok FP16 or 44.23 ms/tok INT4) plus 0.35 ms/tok round-trip DMA and 15.80 ms/tok across the 20 GPU layers ($\Delta t_{\text{E2E}}^{\text{per-tok}} = t_{\text{prefill}} + N_{\text{tok}}(\delta t_{\text{CPU},k=1} + \delta t_{\text{DMA}} + \delta t_{\text{GPU,20L}})$).

Offloading Execution Regime ($n = 50$ Runs)	DMA Dispatch t_{dispatch} (ms)	Raw CPU Slice Δt_{CPU} (ms)	Useful GPU Overlap $\Delta t_{\text{GPU_useful}}$ (ms)	Host Sync Wait $\Delta t_{\text{sync_wait}}$ (ms)	End-to-End Latency Δt_{E2E} (ms)	Speedup vs. INT4 Per-Tok	Speedup vs. FP16 Per-Tok
Standard Per-Token CPU Offload (FP16, $N_{\text{tok}} = 160$, $k = 1$, 8L)	$0.35 \times 160 = 56.0$	$98.25 \times 160 = 15,720.0$	0.00 ms (Serial stall)	15,776.0 ms (98.6/tok)	$18,594.4 \pm 89.9$ ms	0.53×	1.00×
Standard Per-Token CPU Offload (INT4 Split, $N_{\text{tok}} = 160$, $k = 1$, 8L)	$0.35 \times 160 = 56.0$	$44.23 \times 160 = 7,076.8$	0.00 ms (Serial stall)	7,132.8 ms (44.6/tok)	$9,852.9 \pm 70.1$ ms	1.00×	1.89×
Single-Shot Synchronous Blocking (INT4, No Overlap, $k = 4$, 5L)	0.194 ± 0.164	46.04 ± 0.71	0.00 ms (GPU idle wait)	45.712 ± 0.622	$2,927.0 \pm 19.0$ ms	3.37×	6.35×
Single-Shot Async Overlapped (FP16 Slice, $k = 4$, 5L)	0.194 ± 0.164	125.12 ± 0.83	209.39 ± 2.15	0.015 ± 0.033	$2,881.2 \pm 18.6$ ms	3.42×	6.45×
Single-Shot Async Overlapped (INT4 Split, $k = 4$, 5L, Ours)	$\mathbf{0.194 \pm 0.164}$	$\mathbf{46.04 \pm 0.71}$	$\mathbf{209.39 \pm 2.15}$	$\mathbf{0.0135 \pm 0.0079}$	$\mathbf{2,881.3 \pm 18.9}$ ms	**3.42**×	**6.45**×

Percentile Distribution ($n = 50$, INT4 Async Overlapped): t_{dispatch}: median 0.146 ms (P95 0.300 ms); $\Delta t_{\text{CPU (INT4)}}$: median 46.01 ms (P95 47.09 ms); $\Delta t_{\text{CPU (FP16)}}$: median 125.12 ms (P95 126.78 ms), giving $\mathbf{125.12/46.04 = 2.72}\times$ raw CPU speedup; $\Delta t_{\text{GPU_useful}}$: median 209.42 ms (P95 212.59 ms); $\Delta t_{\text{sync_wait}}$: median 0.0118 ms (P95 0.0179 ms).

6.1 Reconciled Per-Token vs. Single-Shot Overlapped Latency ($n = 50$)

Table 5 reconciles the exact arithmetic between per-token layer offloading and our single-shot asynchronous pipeline across $n = 50$ hardware runs:

- **Why Per-Token Offloading Scales with $N_{\text{tok}} = 160$ at $k = 1$:** In standard layer offloading (`accelerate` / `llama.cpp`), the CPU executes all 8 offloaded layers $\mathcal{L}_{12..19}$ on a single token vector ($k = 1$) on every autoregressive decoding step. For $k = 1$ across 8 layers, raw CPU latency is $\delta t_{\text{CPU},k=1}^{\text{INT4}} = 44.23$ ms/tok in INT4 Split and $\delta t_{\text{CPU},k=1}^{\text{FP16}} = 98.25$ ms/tok in FP16 (consistent with the $k = 1$ FP16 measurement of 98.4 ms in Table 6b). Adding $\delta t_{\text{DMA}} = 0.35$ ms/tok round-trip PCIe transfer and $\delta t_{\text{GPU,20L}} = 15.80$ ms/tok for the 20 on-GPU layers yields 60.38 ms/tok (INT4) and 114.40 ms/tok (FP16). Over $N_{\text{tok}} = 160$ generated tokens plus prefill (192.2 ms INT4; 290.0 ms FP16), total end-to-end latency is:

$$\Delta t_{\text{E2E}}^{\text{INT4-tok}} = 192.2 + 160 \times 60.38 = \mathbf{9,852.9} \text{ ms} \quad (8)$$

$$\Delta t_{\text{E2E}}^{\text{FP16-tok}} = 290.0 + 160 \times 114.40 = \mathbf{18,594.4} \text{ ms} \quad (9)$$

- **Single-Shot Overlap Speedup Ratios:** Running our $k = 4$, 5-routed-layer INT4 CPU pass (46.04 ± 0.71 ms, which is $125.12/46.04 = \mathbf{2.72}\times$ faster than the 125.12 ms FP16 pass) synchronously without overlap yields $\Delta t_{\text{E2E}} = 2,927.0$ ms (45.71 ms GPU idle stall). Overlapping the CPU pass inside the 209.39 ms concurrent GPU window reduces host synchronization wait to 0.0135 ms (10–14 μs), achieving $\Delta t_{\text{E2E}} = \mathbf{2,881.3}$ ms—an exact $\mathbf{3.42}\times$ speedup over INT4 per-token offloading ($9,852.9/2,881.3$) and a $\mathbf{6.45}\times$ speedup over FP16 per-token offloading ($18,594.4/2,881.3$).

6.2 Layer Depth Sensitivity & Bottleneck Capacity ($k \in \{1, 2, 4, 8, 16\}$)

Table 6(a) evaluates single-shot steering injection at Early ($\mathcal{L}_6$, 21.4% depth), Middle ($\mathcal{L}_{14}$, 50.0% depth), and Late ($\mathcal{L}_{22}$, 78.6% depth) layers on `Systems-100`. Injection at $\mathcal{L}_{14}$ achieves the highest held-out cosine alignment (0.9903), lowest gold perplexity (4.64), and highest downstream accuracy (61.0% unsupervised SVD / 96.0% calibrated), empirically confirming our middle-layer partitioning hypothesis.

Table 6(b) sweeps the Context Funnel bottleneck capacity across $k \in \{1, 2, 4, 8, 16\}$ vectors (3 KB to 48 KB). Aggressive compression to $k = 1$ or $k = 2$ loses multi-entity relationships, dropping calibrated accuracy to 72.0% and 85.0%. Conversely, expanding to $k = 16$ (48 KB) yields negligible accuracy gain (97.0% vs. 96.0% at $k = 4$) while increasing FP16 CPU execution time to 254.3 ms—overflowing the 209.39 ms concurrent GPU window. Thus, $k = 4$ (12 KB, 46.0 ms INT4) represents the exact Pareto knee.

Table 6: **Ablation Study (`Systems-100`): (a) Layer Depth Sensitivity ($\mathcal{L}_6, \mathcal{L}_{14}, \mathcal{L}_{22}$) and (b) Funnel Bottleneck Capacity Sweep ($k \in \{1, 2, 4, 8, 16\}$ Vectors).** Demonstrating that $\mathcal{L}_{14}$ (50% depth) maximizes subspace alignment and that $k = 4$ (12 KB) sits at the exact Pareto knee where CPU latency remains safely inside the 209.39 ms GPU overlap window.

(a) Injection Depth Sensitivity Ablation ($k = 4$, 12 KB)

Injection Depth	Held-Out $\cos\theta$	Gold PPL ↓	Unsup. SVD Acc ($\mathcal{L}_{\text{CE}} = 0$)	Calibrated Acc (%)
Early ($\mathcal{L}_6$, 21.4%)	0.9512	8.52	44.0%	76.0%
Mid ($\mathcal{L}_{14}$, 50.0%)	**0.9903**	**4.64**	**61.0%**	**96.0%**
Late ($\mathcal{L}_{22}$, 78.6%)	0.9648	6.65	49.0%	81.0%

Note: Early $\mathcal{L}_6$ lacks multi-hop binding; Late $\mathcal{L}_{22}$ *is already committed to surface token logits.*

(b) Funnel Bottleneck Capacity Sweep ($k \in \{1, 2, 4, 8, 16\}$ at $\mathcal{L}_{14}$)

Vectors k	Payload (KB)	Held-Out $\cos\theta$	CPU FP16 (ms)	CPU INT4 (ms)	Unsup. Acc	Calib. Acc
$k = 1$ ($[t_{-1}]$)	3 KB	0.9412	98.4 ms	38.2 ms	48.0%	72.0%
$k = 2$ ($[\mu, t_{-1}]$)	6 KB	0.9748	109.6 ms	41.5 ms	55.0%	85.0%
$k = 4$ **(Pareto Knee)**	**12** KB	**0.9903**	**125.1** ms	**46.0** ms	**61.0%**	**96.0%**
$k = 8$ (4 seg + tail)	24 KB	0.9918	168.7 ms	62.4 ms	62.0%	96.0%
$k = 16$ (12 seg + tail)	48 KB	0.9924	254.3 ms‡	96.8 ms	62.0%	97.0%

‡At $k = 16$ (48 KB), full FP16 CPU execution (254.3 ms) overflows the 209.39 ms concurrent GPU compute window, re-introducing synchronous GPU stalls ($\sim$ 45 ms) for only +1% accuracy gain over $k = 4$.

Table 7: **Expanded $N = 200$ Paired Statistical Evaluation Across `Systems-100` (100 Parametric Distributed Systems Instances Across 10 Templates) and `GSM8K-100` (100 Multi-Step Word Problems).** Because marginal 95% Wilson confidence intervals overlap between baseline and unsupervised bridges, we report **Paired Accuracy Gains (ΔAcc)**, **95% Paired Bootstrap CIs**, **95% Template-Clustered Block Bootstrap CIs** (resampling the 10 template families on `Systems-100`), and **Exact Two-Sided Paired McNemar p-Values** against the GPU Standard CoT baseline.

Evaluation Regime & Bridge Configuration	QA / RAM Footprint	`Systems-100` ($N = 100$, 10 Template Clusters ×10) Acc (%)	95% Wilson Interval	Paired ΔAcc [Paired / Clustered 95% CI]	McNemar Exact p	`GSM8K-100` ($N = 100$ Standardized Math) Acc (%)	95% Wilson Interval	Paired ΔAcc [Paired 95% CI]	McNemar Exact p
Category I: Unsupervised / Zero-QA Regimes *(0 QA Labels, $\mathcal{L}_{CE} = 0.0000$)*									
Baseline 1.5B Standard CoT (GPU Only)	0 / 0 MB	49.0%	[39.4%, 58.7%]	Baseline (0.0%)	—	53.0%	[43.3%, 62.5%]	Baseline (0.0%)	—
Same-Model Bridge (W = I, 0 Prompts)	**0 / 0** MB	**63.0**%	[53.2%, 71.8%]	**+14.0**% [+6.0, +23.0] / [+6.0, +22.0]	**0.0013**	**66.0**%	[56.3%, 74.5%]	**+13.0**% [+6.0%, +21.0%]	**0.0023**
Cross-Scale 500p Answer-Free SVD	**0 / 808** MB	**61.0**%	[51.2%, 70.0%]	**+12.0**% [+4.0, +21.0] / [+3.0, +20.0]	**0.0075**	**64.0**%	[54.2%, 72.7%]	**+11.0**% [+3.0%, +19.0%]	**0.0127**
Category II: Supervised / Task-Calibrated Subspace Refinement *($N_{cal} = 40$ Disjoint Synthetic Examples, $r = 32$ Adapter)*									
Cross-Scale Task-Calibrated Bridge	**40 / 808 MB**	**96.0**%	[90.2%, 98.4%]	**+47.0**% [+37.0, +57.0] / [+41.0, +53.0]	$\mathbf{1.4 \times 10^{-14}}$	**91.0**%	[83.8%, 95.2%]	**+38.0**% [+29.0%, +47.0%]	$\mathbf{7.3 \times 10^{-12}}$

Table 8: **Experiment E: Causal Negative Controls at Layer 14 (⟨/think⟩ Transition Token]).** All injected negative-control vectors are scaled to match the exact ℓ_2 norm of the true subconscious displacement ($\|\Delta_{\text{ctrl}}\|_2 = \|\Delta_{\text{sub}}\|_2 = 39.16$). Random, permuted, and cross-problem deltas fail to improve accuracy ($p \geq 0.109$), proving that gains stem from problem-specific latent geometry rather than perturbation magnitude.

Layer-14 Transition Injection Condition	Control Category	Cosine Sim. w/ True Δ_{sub}	Gold PPL ↓	`Systems-100` ($N = 100$) Acc (%)	Paired ΔAcc [95% CI]	`GSM8K-100` ($N = 100$) Acc (%)	Paired ΔAcc [95% CI]	McNemar Exact p
0. No Injection Baseline (`Standard CoT`, $\alpha = 0$)	Baseline	0.0000	7.76	49.0%	0.0% (Ref)	53.0%	0.0% (Ref)	1.000
1. Norm-Matched Isotropic Gaussian ($\|\Delta_{\text{rand}}\|_2 = \|\Delta_{\text{sub}}\|_2$)	Negative Ctrl	+0.0041	8.19	46.0%	−3.0% [−8.0%, +2.0%]	50.0%	−3.0% [−8.0%, +2.0%]	0.453
2. Feature-Shuffled Permutation ($\text{Permute}_{d=1536}(\Delta_{\text{sub}})$)	Negative Ctrl	−0.0089	8.05	47.0%	−2.0% [−7.0%, +2.0%]	51.0%	−2.0% [−7.0%, +3.0%]	0.688
3. Cross-Problem Mismatched Delta ($\Delta_{\text{sub}}^{(j \neq i)}$ from unrelated query)	Negative Ctrl	+0.1142	8.64	43.0%	−6.0% [−12.0%, +0.0%]	47.0%	−6.0% [−12.0%, +0.0%]	0.109
4. True 500p Answer-Free SVD Delta ($\Delta_{\text{sub}}^{\text{SVD}}$, 0 QA Labels)	**Unsup. Bridge**	**+0.9903**	**5.04**	**61.0**%	**+12.0**% [**+4.0**%, **+20.0**%]	**64.0**%	**+11.0**% [**+3.0**%, **+19.0**%]	**0.0075**
5. True Task-Calibrated Subspace Delta ($\Delta_{\text{sub}}^{\text{Cal}}$, $r = 32$)	**Calib. Bridge**	**+0.9941**	**4.64**	**96.0**%	**+47.0**% [**+37.0**%, **+57.0**%]	**91.0**%	**+38.0**% [**+29.0**%, **+48.0**%]	$\mathbf{1.4 \times 10^{-14}}$

6.3 Paired Statistical Evaluation on `Systems-100` and `GSM8K-100` ($N = 200$)

Table 7 evaluates two 100-problem benchmarks: (1) `Systems-100` (10 randomized parameterizations across each of the 10 distributed systems templates with exact analytical solvers) and (2) `GSM8K-100` (100 standardized grade-school math word problems).

We note that single-method marginal 95% Wilson confidence intervals overlap between the Standard CoT baseline (49.0%, [39.4%, 58.7%]) and the Category I unsupervised bridges (63.0%, [53.2%, 71.8%] for $\mathbf{W} = \mathbf{I}$; 61.0%, [51.2%, 70.0%] for 500-prompt SVD). Because all methods are evaluated on the *exact same problem instances*, marginal Wilson intervals ignore the strong positive pairing across items. We therefore conduct three rigorous paired tests against the Standard CoT baseline:

1. **Exact Paired McNemar's Test:** On `Systems-100`, the Zero-Calibration $\mathbf{W} = \mathbf{I}$ bridge resolves $b = 16$ baseline failures while losing $c = 2$ ($\Delta\text{Acc} = +14.0\%$, $\chi^2_{\text{cc}} = 9.39$, exact two-sided $p = \mathbf{0.0013}$), and the Cross-Scale 500-prompt Answer-Free SVD bridge resolves $b = 15$ while losing $c = 3$ ($\Delta\text{Acc} = +12.0\%$, $\chi^2_{\text{cc}} = 6.72$, exact $p = \mathbf{0.0075}$). On `GSM8K-100`, $\mathbf{W} = \mathbf{I}$ achieves +13.0% ($p = \mathbf{0.0023}$) and 500-prompt SVD achieves +11.0% ($p = \mathbf{0.0127}$). Pooled across all $N = 200$ problems, pure answer-free SVD achieves +11.5% ($p = \mathbf{1.2 \times 10^{-4}}$).

2. **Template-Clustered Block Bootstrap:** Because `Systems-100` instantiates 10 parameterizations per template (`P01`..`P10`), errors within a template family may correlate. Resampling at the *cluster level* (10 template blocks with replacement over 5,000 iterations) yields 95% confidence intervals of [+6.0%, +22.0%] for $\mathbf{W} = \mathbf{I}$ and [+3.0%, +20.0%] for 500-prompt Answer-Free SVD—both strictly excluding zero.

3. **Task-Calibrated Subspace Refinement:** In Category II, rank-32 subspace refinement reaches **96.0**% ($\Delta\text{Acc} = +47.0\%$, clustered CI [+41.0%, +53.0%], $p = 1.4 \times 10^{-14}$) on `Systems-100` and **91.0**% ($\Delta\text{Acc} = +38.0\%$, $p = 7.3 \times 10^{-12}$) on `GSM8K-100`.

6.4 Causal Negative Controls at Layer 14

To verify that accuracy improvements are caused by the specific geometric content of Δ_{sub} rather than implicit regularization from adding noise at Layer 14, **Table 8** evaluates three norm-matched negative controls ($\|\Delta_{\text{ctrl}}\|_2 =$

$\|\Delta_{\text{sub}}\|_2 = 39.16$): (1) isotropic Gaussian noise Δ_{rand}, (2) feature-shuffled coordinates Permute(Δ_{sub}) (preserving exact norm and marginal value distribution while destroying basis alignment), and (3) a valid subconscious delta $\Delta_{\text{sub}}^{(j\neq i)}$ computed from an unrelated problem. All three negative controls slightly degrade accuracy (-2.0% to -6.0%, $p \geq 0.109$) and worsen gold perplexity (8.05–8.64 vs. 7.76 baseline), confirming that downstream gains require directionally aligned latent transfer from the current query.

7 Discussion: Thinking-as-a-Service (TaaS) & Future Work

Because our bridge transmits only $k = 4$ compressed vectors (12 KB) once per query, the Subconscious slice $\mathcal{L}_{12..19}$ does not need to reside on the local motherboard:

- **Privacy-Preserving Edge Personalization:** A user's personal 1.5B model (*the Mouth*) runs 100% locally on an edge GPU. When Layer-14 entropy spikes on a complex reasoning query, the device transmits a 12 KB latent packet to a shared, frozen cloud foundation model slice (*the Brain*). Raw user text never leaves the device, and cloud compute costs drop by $> 50\times$ because the server executes a single non-autoregressive pass rather than streaming hundreds of tokens.

- **Part II — Rolling Latent NCA & Superposition-Embedded RAG:** In follow-up work, we generalize static middle-layer injection into an 8-cell *Rolling Latent Neural Cellular Automata (NCA)* ring governed by PCA manifold snapping and fixed-point attractor dynamics ($\|\mathbf{S}_t - \mathbf{S}_{t-1}\|_2 \to 0$), enabling standard 7B–8B LLMs to skim across thousands of 28 KB pre-encoded data cartridges with $\mathcal{O}(1)$ active working memory.

Acknowledgments and LLM Disclosure

Google Gemini was utilized for LaTeX typesetting, TikZ/PGFPlots diagram generation, table formatting, and manuscript editing. All architectural designs, mathematical formulations, and experimental benchmarks were conceived, executed, and verified by the author.

References

[1] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.

[2] S. Hao et al., "Training large language models to reason in a continuous latent space," *arXiv preprint arXiv:2412.06769*, 2024.

[3] S. Xu et al., "Chain of draft: Thinking faster by writing less," *arXiv preprint arXiv:2502.18600*, 2025.

[4] Y. Sheng et al., "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. International Conference on Machine Learning (ICML)*, pp. 31094–31116, 2023.

[5] P. H. Schönemann, "A generalized solution of the orthogonal Procrustes problem," *Psychometrika*, vol. 31, no. 1, pp. 1–10, 1966.

[6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30318–30332, 2022.

[7] J. Lin et al., "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Machine Learning and Systems (MLSys)*, vol. 6, pp. 87–100, 2024.

[8] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. International Conference on Machine Learning (ICML)*, pp. 19274–19286, 2023.

[9] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pp. 1–15, 2022.